UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

Software Engineering for Embedded Systems
Project work

Real-time Scheduling Simulator

Edoardo Sarri

7173337

Giugno 2025

Indice

1 Introduzione **4**
1.1 Capacità 4

2 Utilizzo **5**
2.1 Baseline 5

3 Analisi **7**
3.1 Componenti 7
3.1.1 Task 7
3.1.2 Job 7
3.1.3 Taskset 7
3.1.4 CPU 7
3.1.5 Scheduler 7
3.2 Class diagram 7

4 Implementazione **9**
4.1 Scheduler 9
4.1.1 Rate Monotonic 10
4.1.2 Earliest Deadline First 11
4.2 Fault injection 11
4.3 Utility 11
4.3.1 Logging 11
4.3.2 Clock 11
4.3.3 Eccezioni 11

5 Dataset **12**
5.1 Dataset EDF 12
5.2 Dataset RM 13

Elenco delle figure

2.1 Baseline. 6

3.1 Class diagram 8

4.1 Sequence Diagram scheduling 10

5.1 Taskset schedulato con EDF 12
5.2 Taskset schedulato con RM 13

1 Introduzione

Il progetto vuole modellare e implementare un sistema in Java che permetta di generare tracce di un'esecuzione a partire dalla definizione di un taskset.

Ogni traccia è definita come una sequenza di coppie $< tempo, evento >$, dove un *evento* può essere: rilascio di un job di un task; esecuzione di un job; completamento di un job; preemption che un task può subire.

1.1 Capacità

A partire da un taskset, il sistema ha le capacità di:

- Generare la traccia di esecuzione del taskset schedulato tramite un dato algoritmo di scheduling (i.e., Rate Monotonic e Earliest Deadline First).
- Generare un dataset di tracce relative a più simulazioni su un taskset.
- Specificare il tempo desiderato della simulazione.
- Rilevare eventuali deadline miss. Se un job non ha finito di eseguire entro la propria deadline allora la simulazione si arresta; non continua perché ci interessa valutare il primo fallimento, visto che i successivi potrebbero essere in cascata del primo.
- Campionare da una distribuzione l'execution time di un job.
- Definito uno scheduler con il relativo taskset, eseguire il controllo di feasibility adeguato per la schedulabilità del taskset. Per RM è stato implementato l'hyperbolic bound; per EDF il controllo necessario e sufficiente sul fattore di utilizzo.

2 Utilizzo

Nei prossimi punti vediamo degli accorgimenti che sono utili per far utilizzare il sistema nel modo corretto. Per osservare degli esempi di come strutturare il codice osservare le sezioni di questo capitolo.

- Gli elementi del sistema, come task, taskset e scheduler, devono essere definiti all'interno del main. Una volta definito tutto si deve chiamare il metodo `schedule` sullo scheduler. Nel caso si voglia generare una sorta di dataset di tracce, è prevista la possibilità di chiamare più volte il metodo `schedule` all'interno dello stesso main. Per generare un dataset di tracce è invece necessario chiamare il metodo `scheduleDataset` passando come parametro il numero di tracce.

- Un task campiona il suo tempo di esecuzione da una distribuzione. Tale distribuzione, passata come parametro del costruttore, è un oggetto di tipo `Sampler` definito nella libreria Sirio. Oltre alle distribuzioni introdotte da Sirio è presente anche l'implementazione `ConstantSampler`, che definisce un campionamento costante.

- I tempi devono essere passati al sistema e letti da esso in millisecondi. Il sistema li gestisce in nanosecondi per avere un'alta precisione.

Per concludere analizziamo come il sistema si comporta sulla generazione di dataset. Si procederà in modo incrementale, per capire come affrontare lo scheduling di taskset più semplici e poi quelli più complessi.

Questa parte può essere anche utile per capire come strutturare il main ed utilizzare il simulatore.

2.1 Baseline

Vediamo come funziona una simulazione tramite RM con un taskset molto semplice. Il taskset in questione, come si può notare dal codice in Figura 2.1a, è molto semplice in modo da poter osservare le differenze tra le due simulazioni. Si tratta di un taskset con due task puramente periodici: il primo ha periodo e deadline relativo pari a 20ms e un execution time di 10ms; il secondo ha periodo e deadline di 60ms e un execution time di 12ms.

La traccia in Figura 2.1b è l'output della simulazione.

```java
Task task1 = new Task(
    20,
    20,
    List.of(
        new Chunk(1, new ConstantSampler(new BigDecimal(10)))));
Task task2 = new Task(
    60,
    60,
    List.of(
        new Chunk(1, new ConstantSampler(new BigDecimal(5))),
        new Chunk(2, new ConstantSampler(new BigDecimal(4))),
        new Chunk(3, new ConstantSampler(new BigDecimal(3)))));
TaskSet taskSet = new TaskSet(Set.of(task1, task2));
Scheduler rm = new RMScheduler(taskSet, 60);
rm.schedule();
```

(a) Taskset della baseline

```
[INFO] <0.000, release Task1>
[INFO] <0.000, release Task2>
[INFO] <0.000, execute Chunk1.1>
[INFO] <10.000, finish Chunk1.1>
[INFO] <10.000, complete Task1>
[INFO] <10.000, execute Chunk2.1>
[INFO] <15.000, finish Chunk2.1>
[INFO] <15.000, execute Chunk2.2>
[INFO] <19.000, finish Chunk2.2>
[INFO] <19.000, execute Chunk2.3>
[INFO] <20.000, release Task1>
[INFO] <20.000, preempt Task2>
[INFO] <20.000, execute Chunk1.1>
[INFO] <30.000, finish Chunk1.1>
[INFO] <30.000, complete Task1>
[INFO] <30.000, execute Chunk2.3>
[INFO] <32.000, finish Chunk2.3>
[INFO] <32.000, complete Task2>
[INFO] <40.000, release Task1>
[INFO] <40.000, execute Chunk1.1>
[INFO] <50.000, finish Chunk1.1>
[INFO] <50.000, complete Task1>
[INFO] <60.000, release Task1>
[INFO] <60.000, release Task2>
[INFO] <60.000, end>
```

(b) Traccia della baseline

Figura 2.1: Baseline.

3 Analisi

In questo capitolo analizziamo la struttura del progetto, partendo dai suoi componenti e definendo la loro relazione.

3.1 Componenti

3.1.1 Task

Un task è il template statico del sistema. È definito da: una deadline; un periodo; un sampler per l'execution tempo; una priorità nominale.

Non ci interessa definire un activation time perché vogliamo considerare il caso pessimo: l'activation time sarà uguale per tutti i task e coinciderà con l'istante iniziale.

3.1.2 Job

Un job rappresenta una singola istanza di esecuzione di un task. È l'entità dinamica che mantiene lo stato durante la simulazione. È definito da: il riferimento al task padre; il tempo di rilascio; la deadline assoluta; il tempo di esecuzione rimanente; la priorità dinamica.

3.1.3 Taskset

È un insieme di task. È l'oggetto principale gestito dallo scheduler.

3.1.4 CPU

È l'unità di elaborazione. Supponiamo essere unica.

3.1.5 Scheduler

È il componente che assegna un job al processore e gestisce l'avanzamento del tempo e il logging. Sono stati implementati Rate Monotonic (RM) e Earliest Deadline First (EDF).

3.2 Class diagram

Per capire meglio la struttura del progetto, analizziamo il diagramma delle classi in Figura 3.1.

In questo modello sono definiti solo i campi che definiscono i vari componenti; per chiarezza del sistema non sono stati infatti introdotti gli oggetti necessari all'implementazione.

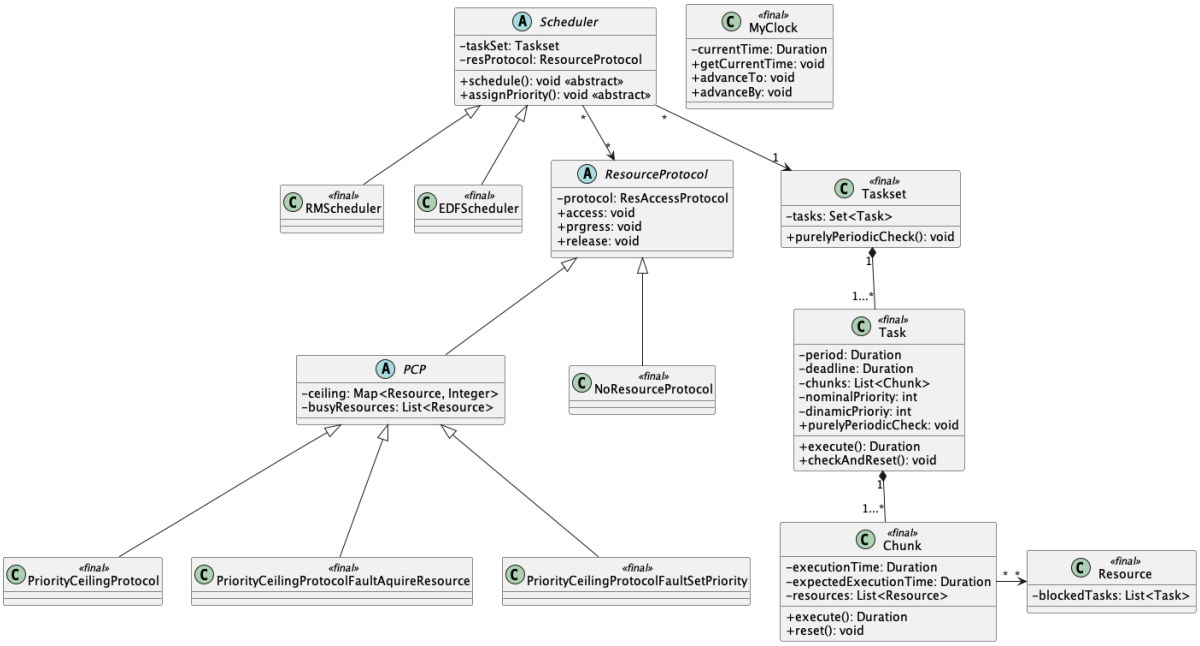

Figura 3.1: Class diagram

4 Implementazione

L'implementazione deve rappresentare una simulazione deterministica. Per questo motivo non sono stati usati thread (oggetti Java `Thread`), la gestione del tempo avviene in modo discreto ed è centralizzata nello scheduler.

4.1 Scheduler

Ogni possibile implementazione di un algoritmo di scheduling è un'estensione della classe `Scheduler`. Questo permette di definire i comportamenti comuni a tutti gli scheduler e di astrarre future implementazioni.

Per l'implementazione è stato necessario mantenere una lista di job pronti `readyTasks` e una di job bloccati `blockedTask`. Inoltre è necessario mantenere un riferimento all'ultimo job che è andato in esecuzione.

L'implementazione della logica di scheduling è la stessa sia per RM che per EDF: il metodo `schedule` con i metodi helper necessari sono stati, per questo motivo, inseriti all'interno della classe base. Ogni implementazione concreta deve solo definire alcuni aspetti usati poi nella logica. Per modellare questo principio è stato usato il Template Method pattern: il metodo `schedule` è dichiarato pubblico e `final` in modo che non possa essere modificato dalle implementazioni, mentre gli hooks chiamati al suo interno sono dichiarati `abstract` e `protected`.

Per gestire i tempi rilevanti, cioè quelli in cui lo scheduler deve prendere il controllo del sistema per rivedere la propria politica di scheduling, è delegata alla struttura `readyTasks`. Gli eventi rilevanti sono l'unione ordinata dei multipli di ciascun periodo fino alla durata della simulazione.

Per capire i passaggi che esegue la simulazione consideriamo e analizziamo il sequence diagram di Figura 4.1:

- `assignPriority`
  Assegna le priorità nominali ai task. È dichiarato astratto in `Scheduler` e deve essere implementato dalle classi concrete.

- `MyClock.reset`
  Resetta il clock di sistema prima di iniziare la simulazione.

- `initStructure`
  Inizializza le strutture dati: la lista di job pronti e la mappa dei job attivi per task. Rilascia i primi job al tempo 0.

- `checkFeasibility`
  Controlla la schedulabilità del taskset.

- `executeFor`
  Metodo dello scheduler che manda in esecuzione il job a priorità maggiore per un intervallo di tempo. Gestisce l'avanzamento del clock e il logging degli eventi di esecuzione, preemption e completamento.

- `Job.execute`
  Riduce il tempo di esecuzione rimanente del job.

- `releasePeriodTasks`
  Si occupa di rilasciare i nuovi job allo scoccare del periodo di ogni task e controlla che il job precedente abbia terminato, altrimenti solleva una `DeadlineMissException`.

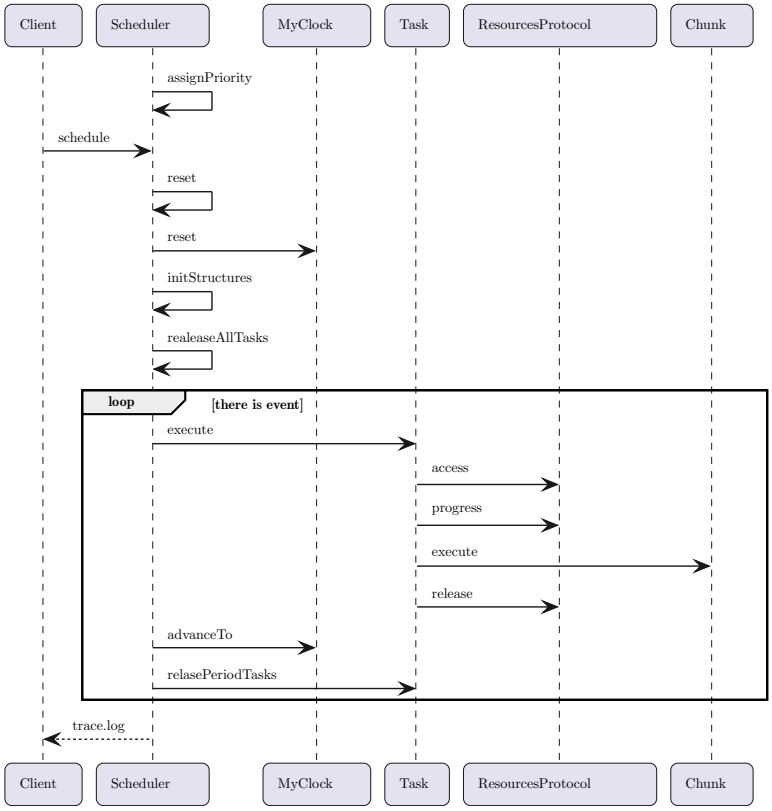

Figura 4.1: Sequence Diagram scheduling

4.1.1 Rate Monotonic

Vediamo come sono implementati i metodi astratti della classe `Scheduler` in RM.

- `checkFeasibility`
  Viene controllato l'hyperbolic bound.

- `assignPriority`
  Assegna la priorità in modo inverso rispetto alla durata del periodo.
- `addReadyTask`
  Aggiunge il job alla lista ordinata secondo la priorità dinamica.

4.1.2 Earliest Deadline First

Vediamo come sono implementati i metodi astratti della classe `Scheduler` in EDF.

- `checkFeasibility`
  Viene controllato che il fattore di utilizzo del taskset sia minore o uguale a 1.
- `assignPriority`
  Assegna la priorità in modo inverso rispetto alla durata della deadline.
- `addReadyTask`
  Aggiunge il job alla lista e la riordina secondo la deadline assoluta del job.

4.2 Fault injection

Il sistema permette di modellare fault legati al tempo di esecuzione. Essendo il tempo di esecuzione campionato da una distribuzione fornita al task, è possibile modellare comportamenti stocastici che superano il caso medio o il WCET previsto.

4.3 Utility

4.3.1 Logging

Il logging è centralizzato nello scheduler e scrive su un file una sequenza di coppie $< tempo, evento >$. Il file di destinazione è `trace.log`.

4.3.2 Clock

Il tempo è gestito globalmente tramite il Singleton `MyClock`. Lo scheduler è l'unica entità responsabile di far avanzare il clock tramite i metodi `advanceTo` e `advanceBy`. Il sistema gestisce il tempo in nanosecondi per garantire precisione, esponendolo in millisecondi all'utente.

4.3.3 Eccezioni

- `DeadlineMissedException`
  Viene sollevata dallo scheduler quando un job supera la propria deadline o quando un nuovo job viene rilasciato prima che il precedente sia terminato.

5 Dataset

In questo capitolo si presentano le simulazioni ottenute da due taskset con una complessità maggiore rispetto a quelli del Capitolo 4.

5.1 Dataset EDF

Il primo taskset complesso, mostrato in Figura 5.1 è composto da 5 task senza risorse condivise ed è quindi schedulato da Earliest Deadline First (EDF). Il taskset in questione ha un fattore di utilizzo della CPU di 0,95 e solo due task sono puramente periodici. Per introdurre la stocasticità ed aumentare la complessità della simulazione si campiona l'execution time dei chunk.

```
Task task1 = new Task(
    40,
    30,
    List.of(
        new Chunk(
            1,
            new ConstantSampler(new BigDecimal(4)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(2))),
        new Chunk(
            2,
            new ConstantSampler(new BigDecimal(4)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(1))),
        new Chunk(
            3,
            new ConstantSampler(new BigDecimal(4)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(1)))));
Task task2 = new Task(
    50,
    40,
    List.of(
        new Chunk(
            1,
            new ConstantSampler(new BigDecimal(4)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(1))),
        new Chunk(
            2,
            new ConstantSampler(new BigDecimal(5)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(1)))));
Task task3 = new Task(
    80,
    80,
    List.of(
        new Chunk(
            1,
            new ConstantSampler(new BigDecimal(7)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(2)))));
Task task4 = new Task(
    100,
    100,
    List.of(
        new Chunk(
            1,
            new ConstantSampler(new BigDecimal(6)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(3))),
        new Chunk(
            2,
            new ConstantSampler(new BigDecimal(6)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(3)))));
Task task5 = new Task(
    60,
    50,
    List.of(
        new Chunk(
            1,
            new ConstantSampler(new BigDecimal(5)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(1))),
        new Chunk(
            2,
            new ConstantSampler(new BigDecimal(5)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(2))),
        new Chunk(
            3,
            new ConstantSampler(new BigDecimal(5)),
            new UniformSampler(new BigDecimal(-1), new BigDecimal(1)))));
TaskSet taskSet = new TaskSet(Set.of(task1, task2, task3, task4, task5));
Scheduler edf = new EDFScheduler(taskSet, 500);
edf.scheduleDataset(10);
```

Figura 5.1: Taskset schedulato con EDF

Considerando la lunghezza delle tracce generate dalla simulazione, non si è potuto inserire il file `trace.log` in questo documento. Nel repository su GitHub è pre-

sente una cartella `output`, nella quale si può comunque osservare il risultato nel file `traceEDF.log`.

5.2 Dataset RM

Il secondo taskset complesso, mostrato in Figura 5.2, è composto da cinque task ed è schedulato tramite Rate Monotonic. Per introdurre la stocasticità ed aumentare la complessità della simulazione si campiona l'execution time di ciascun job da una distribuzione uniforme.

```
Resource res1 = new Resource();
Resource res2 = new Resource();
Task task1 = new Task(
    35,
    35,
    List.of(
        new Chunk(
            1,
            new UniformSampler(new BigDecimal(2), new BigDecimal(2))),
        new Chunk(
            2,
            new UniformSampler(new BigDecimal(1), new BigDecimal(1.5)),
            List.of(res1)),
        new Chunk(
            3,
            new UniformSampler(new BigDecimal(0.5), new BigDecimal(1)))));
Task task2 = new Task(
    50,
    50,
    List.of(
        new Chunk(
            1,
            new UniformSampler(new BigDecimal(3), new BigDecimal(4))),
        new Chunk(
            2,
            new UniformSampler(new BigDecimal(3), new BigDecimal(3.5)),
            List.of(res2)),
        new Chunk(
            3,
            new UniformSampler(new BigDecimal(3), new BigDecimal(3.5)))));
Task task3 = new Task(
    80,
    80,
    List.of(
        new Chunk(
            1,
            new UniformSampler(new BigDecimal(4), new BigDecimal(5))),
        new Chunk(
            2,
            new UniformSampler(new BigDecimal(4), new BigDecimal(4.5)))));
Task task4 = new Task(
    110,
    110,
    List.of(
        new Chunk(
            1,
            new UniformSampler(new BigDecimal(5), new BigDecimal(5.5))),
        new Chunk(
            2,
            new UniformSampler(new BigDecimal(2), new BigDecimal(3)),
            List.of(res2))));
Task task5 = new Task(
    160,
    160,
    List.of(
        new Chunk(
            2,
            new UniformSampler(new BigDecimal(3.5), new BigDecimal(4)),
            List.of(res1))));
TaskSet taskSet = new TaskSet(Set.of(task1, task2, task3, task4, task5));
ResourcesProtocol pcp = new PriorityCeilingProtocolFaultAquireResource(0.2);
Scheduler rm = new RMScheduler(taskSet, pcp, 1500);
rm.scheduleDataset(10);
```

Figura 5.2: Taskset schedulato con RM

Come per il caso precedente il file di log relativo a questa simulazione è nella cartella output sul repository di GitHub nel file `traceRM.log`.

Bibliografia

[1] Laura Carnevali. *Appunti slides Software Engineering for Embedded System*.

[2] *chatGTP*.

[3] *documentazione Java, oracle*. https://docs.oracle.com/javase/8/docs/api/overview-summary.html.